DISRUPTIVE ARCHITECTURES
IoT, Big Data e IA

Pipeline de Dados com IoT e Docker

Leituras de temperatura: da ingestão à visualização

Jacqueline dos Santos Vivot
RA 125866

Projeto acadêmico • Outubro de 2026

Python / pandas / SQLAlchemy
PostgreSQL / Docker Compose
Streamlit / Plotly

Fonte: Temperature Readings: IoT Devices, disponibilizada por Atul Anand Jha no Kaggle.

1. Contextualização e proposta

Dispositivos de Internet das Coisas geram observações que precisam ser organizadas para apoiar decisões. Arquivos brutos, isoladamente, dificultam consultas, comparação de ambientes e identificação de variações. O projeto implementa um pipeline em lote para validar leituras históricas de temperatura, armazená-las em um banco relacional e apresentá-las em gráficos interativos.

Fluxo: CSV do Kaggle → tratamento com pandas → carga via SQLAlchemy → PostgreSQL → três views SQL → dashboard Streamlit com Plotly.

O Docker Compose descreve três serviços: banco, processamento e dashboard. Uma verificação de saúde permite iniciar o processamento somente quando o PostgreSQL está pronto; o dashboard depende da conclusão bem-sucedida da carga. Um volume preserva os dados entre reinicializações.

O escopo é descritivo e usa um arquivo histórico. Não existe sensor conectado ao vivo nem modelo treinado de inteligência artificial. Embora a arquitetura seja uma introdução a pipelines de dados, o volume da base não caracteriza, por si só, uma solução distribuída de Big Data.

2. Fonte e modelo de dados

A base possui as colunas id, room_id/id, noted_date, temp e out/in. O identificador original da leitura se torna a chave primária reading_id; room_id/id é mapeado para device_id; noted_date se torna observed_at; temp, temperature; e out/in, location.

O nome device_id é usado para compatibilidade com o exemplo da disciplina, mas a fonte contém apenas o valor Room Admin. Não é possível inferir quantos sensores físicos foram utilizados. In e Out distinguem ambientes internos e externos.

As datas são interpretadas explicitamente como dia-mês-ano, hora e minuto, sem fuso informado. A análise mantém os horários da fonte. A tabela registra temperaturas numéricas e restringe o ambiente aos valores In e Out.

3. Implementação e reprodução

Preparação. Foi criado um ambiente Python isolado e instaladas as dependências. A base real foi baixada do Kaggle. As versões diretas utilizadas estão registradas em requirements.txt.

Validação. O pipeline verifica as colunas, normaliza espaços nos identificadores, converte datas e temperaturas e rejeita campos ausentes, temperaturas não finitas e categorias desconhecidas. Valores extremos válidos são preservados para análise.

Duplicidade. Uma repetição idêntica de ID foi removida. IDs repetidos com informações divergentes interrompem a carga para evitar escolha silenciosa de uma versão. Leituras diferentes feitas no mesmo minuto são preservadas.

Persistência. A carga usa tabela temporária e uma transação PostgreSQL. INSERT ON CONFLICT atualiza IDs existentes e insere novos. Um bloqueio serializa cargas concorrentes. A reexecução do mesmo arquivo não deve aumentar o total de registros.

Visualização. O Streamlit consulta as três views. Há filtro de ambiente, indicadores de quantidade e temperatura, gráficos interativos e exportação do resumo diário. As médias gerais são ponderadas pelo número de leituras.

Docker. Para reproduzir, instale Docker Desktop com WSL 2, salve o CSV em data/IOT-temp.csv, copie .env.example para .env e defina a senha. Execute docker compose up --build -d e acesse http://localhost:8501. Os detalhes e comandos de verificação estão no README.

A configuração Docker foi preparada, mas sua execução neste computador aguarda instalação do Docker Desktop e WSL, adiada por decisão da usuária. A validação local de PostgreSQL, quando registrada, não equivale a um teste em contêiner.

4. Views SQL e finalidade

avg_temp_por_dispositivo

Agrupa por device_id e location. Calcula AVG(temperature) e COUNT(*). Permite comparar as médias dos ambientes interno e externo, além de verificar o número de observações que sustenta cada média. Não compara diferentes sensores físicos nesta base.

leituras_por_hora

Extrai a hora de observed_at e agrupa por hora e location. COUNT(*) mostra o volume acumulado em cada uma das 24 horas ao longo de todo o período. É útil para examinar a concentração da coleta; não mede temperatura nem representa 24 horas consecutivas.

temp_max_min_por_dia

Converte observed_at para DATE e agrupa por data e location. Calcula MAX, MIN e AVG da temperatura. Permite observar amplitudes diárias e investigar dias com extremos. Dias sem registros não devem ser tratados como temperatura zero.

As views são consultas armazenadas, calculadas sobre os registros disponíveis a cada consulta. Não duplicam fisicamente a base como uma tabela materializada. As definições completas ficam em sql/views.sql.

Exemplo de conferência:

```
SELECT COUNT(*) FROM temperature_readings;
SELECT * FROM avg_temp_por_dispositivo;
```

5. Resultados e interpretação

A versão analisada contém 97.606 linhas de origem, nenhuma linha inválida e um ID duplicado. Após a limpeza, restam **97.605 leituras**, entre **28/07/2018 07:06 e 08/12/2018 09:30**.

Ambiente	Leituras	Média	Mínima	Máxima
Interno	20.345	30,45 °C	21 °C	41 °C
Externo	77.260	36,27 °C	24 °C	51 °C

Diferença de médias: o ambiente externo apresenta média aproximadamente 5,81 °C superior à do interno. As amostras possuem tamanhos diferentes e não necessariamente correspondem aos mesmos instantes. A comparação não demonstra causalidade nem eficiência de climatização.

Extremos: a leitura máxima externa é 51 °C; a interna, 41 °C. Em uma aplicação real, esses valores justificariam investigação sobre ambiente, instalação e calibração do sensor. Não são automaticamente erros ou evidência de risco operacional.

Aplicações: acompanhamento de condições de armazenamento, apoio à climatização e alertas com limites definidos por especialistas do contexto. A contagem por hora ajuda a avaliar a distribuição da coleta; a variação diária apoia a investigação de eventos.

Evoluções: coletar via MQTT, incluir identificação física e calibração dos sensores, padronizar fuso horário, monitorar qualidade e testar modelos de anomalias com separação temporal de treino e avaliação. Para volumes maiores, avaliar ingestão em blocos e particionamento do banco.

6. Evidências e validação

Os testes automatizados de transformação verificam formato de datas, descarte de valores não finitos, remoção de duplicatas, rejeição de IDs conflitantes e ausência de colunas obrigatórias. O processamento real confirmou os totais apresentados neste relatório.

A integração com PostgreSQL local foi executada: duas cargas mantiveram 97.605 registros; as somas das views de média e hora conferiram com a tabela; e as mínimas diárias não excederam as máximas. Detalhes em docs/integracao.json.

Dashboard: 01-visao-geral



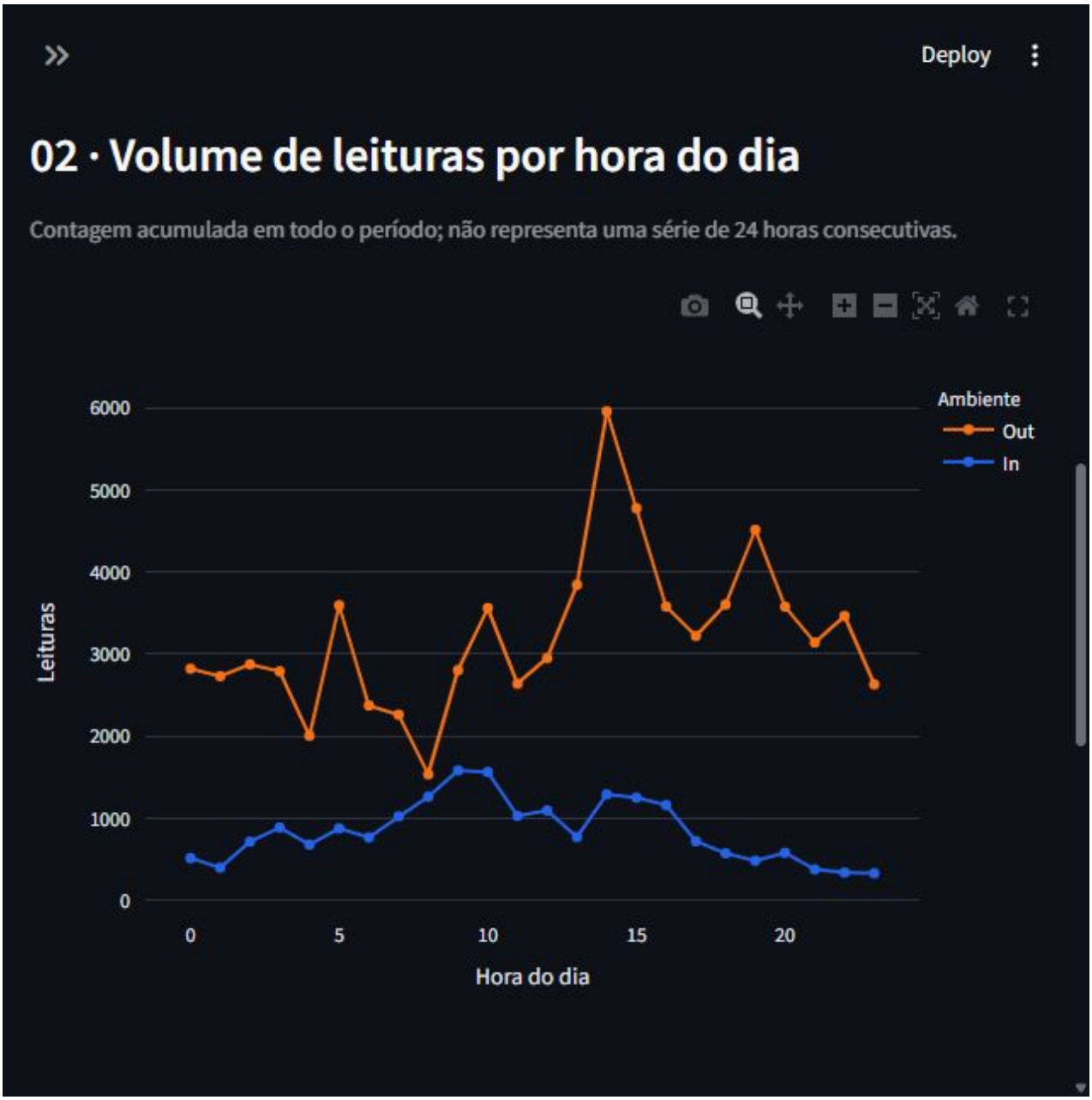
Captura real do Streamlit consultando as views em PostgreSQL local. Execução em Docker pendente neste PC.

Dashboard: 02-media



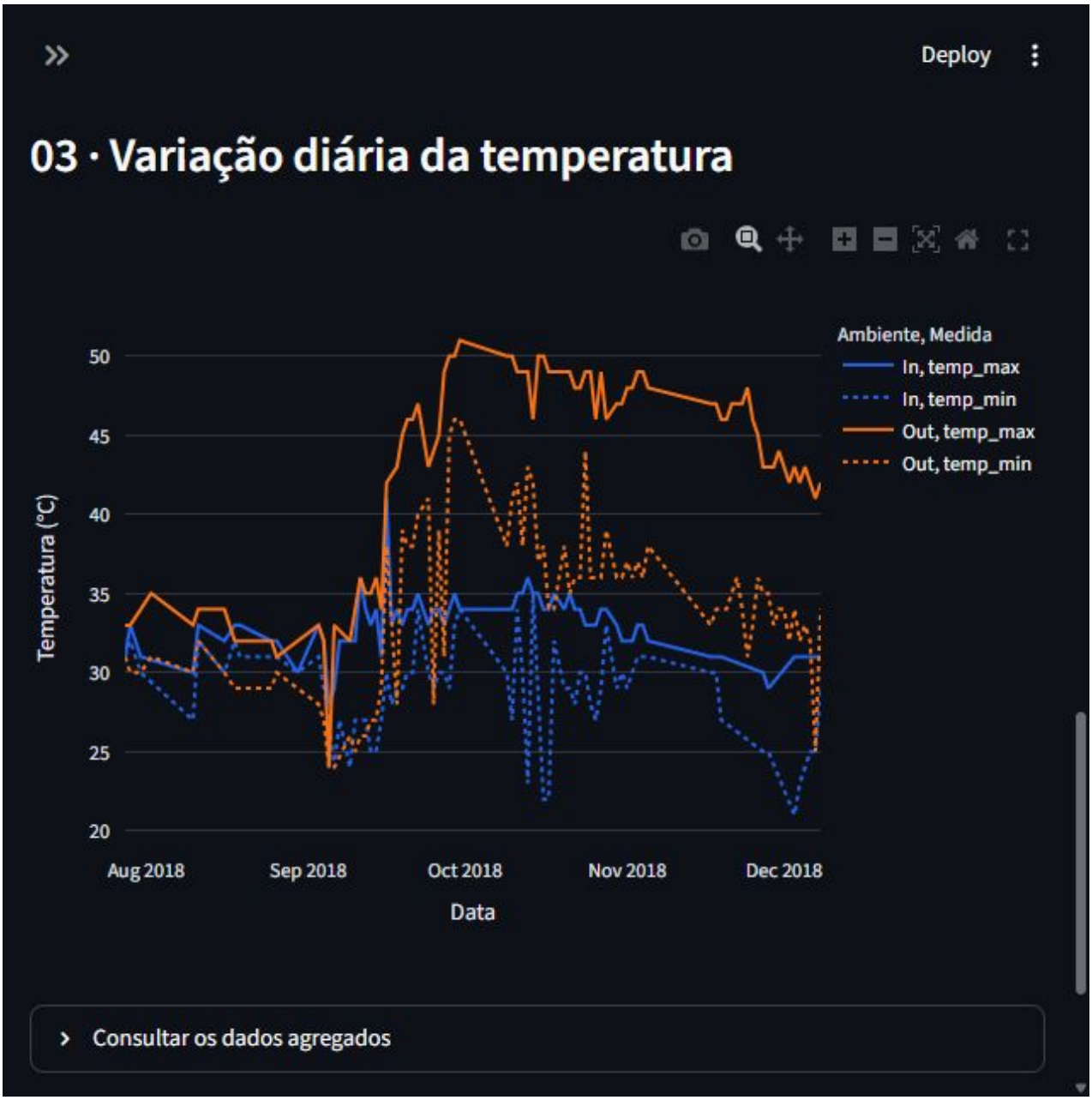
Captura real do Streamlit consultando as views em PostgreSQL local. Execução em Docker pendente neste PC.

Dashboard: 03-horas



Captura real do Streamlit consultando as views em PostgreSQL local. Execução em Docker pendente neste PC.

Dashboard: 04-diario



Captura real do Streamlit consultando as views em PostgreSQL local. Execução em Docker pendente neste PC.

7. Referências e entrega

Atul Anand Jha. Temperature Readings: IoT Devices. Kaggle
https://www.kaggle.com/datasets/atulanandjha/temperature-readings-iot-devices

Docker. Instalação do Docker Desktop no Windows
https://docs.docker.com/desktop/setup/install/windows-install/

PostgreSQL. Documentação
https://www.postgresql.org/docs/

pandas. Documentação
https://pandas.pydata.org/docs/

SQLAlchemy. Documentação
https://docs.sqlalchemy.org/

Streamlit. Documentação
https://docs.streamlit.io/

Plotly. Gráficos em Python
https://plotly.com/python/

Material orientador: enunciado da disciplina fornecido pela estudante, 'Pipeline de Dados com IoT e Docker'.

Repositório: github.com/jacqueline-vivot/Pipeline-de-Dados-com-IoT-e-Docker

Pendências para envio: executar e conferir a versão Docker; confirmar publicação e acesso público do repositório; gravar e publicar o vídeo de até quatro minutos. O roteiro está em docs/roteiro-video.md.